

INTRODUCTION TO LOGIC I

JOONA PIIRAINEN

ABSTRACT. Notes for the University of Helsinki course MAT21014, taught in Finnish from 03.09.2026–17.10.2026.

CONTENTS

1. Course information	1
2. Lecture notes	1
2.1. 03.09.2026	1
2.2. 07.09.2026	2
2.3. 10.09.2026	5
2.4. 14.09.2026	7
2.5. 21.09.2026	8
2.6. 28.09.2026	11

1. COURSE INFORMATION

The official course information is available on the University of Helsinki course page.

2. LECTURE NOTES

2.1. 03.09.2026.

2.1.1. *Propositional formulas.*

Definition 2.1. The set of *propositional formulas* is defined inductively as follows:

- (1) Every propositional symbol p_n , where $n \in \mathbb{N}$, is a propositional formula.
- (2) If A and B are propositional formulas, then

$$\neg A, \quad (A \wedge B), \quad (A \vee B), \quad (A \rightarrow B), \quad (A \leftrightarrow B)$$

are propositional formulas.

- (3) No other strings are propositional formulas.

2.1.2. *Subformulas.*

Definition 2.2 (Immediate subformulas). The immediate subformulas of a propositional formula are defined by its outermost connective:

- (1) A propositional symbol p_n has no immediate subformulas.
- (2) The only immediate subformula of $\neg A$ is A .
- (3) The immediate subformulas of $(A \circ B)$ are A and B , where

$$\circ \in \{\wedge, \vee, \rightarrow, \leftrightarrow\}.$$

Definition 2.3 (Subformulas). A formula B is a *subformula* of a formula A if $B = A$, or if there are formulas

$$A = A_0, A_1, \dots, A_k = B$$

such that $k \geq 1$ and A_{i+1} is an immediate subformula of A_i for every $0 \leq i < k$.

2.1.3. Structural induction.

Theorem 2.4 (Structural induction). *Let $P(A)$ be a statement about propositional formulas. Suppose that*

- (1) $P(p_n)$ holds for every $n \in \mathbb{N}$;
- (2) if $P(A)$ holds, then $P(\neg A)$ holds; and
- (3) if $P(A)$ and $P(B)$ hold, then $P((A \circ B))$ holds for every

$$\circ \in \{\wedge, \vee, \rightarrow, \leftrightarrow\}.$$

Then $P(A)$ holds for every propositional formula A .

Example 2.5 (Structural induction). For a formula A , let $v(A)$ be the number of occurrences of propositional symbols in A , and let $b(A)$ be the number of occurrences of binary connectives in A . We prove that

$$v(A) = b(A) + 1$$

for every propositional formula A .

If $A = p_n$, then $v(A) = 1$ and $b(A) = 0$, so the claim holds. Suppose next that the claim holds for a formula A . Negation introduces neither a propositional symbol nor a binary connective, and hence

$$v(\neg A) = v(A) = b(A) + 1 = b(\neg A) + 1.$$

Finally, suppose that the claim holds for formulas A and B , and let $\circ$ be any binary connective. Then

$$\begin{aligned} v((A \circ B)) &= v(A) + v(B) \\ &= (b(A) + 1) + (b(B) + 1) \\ &= b((A \circ B)) + 1. \end{aligned}$$

The result follows by structural induction. □

2.2. 07.09.2026.

2.2.1. Structural induction.

Example 2.6 (Matching parentheses). For a propositional formula A , let $l(A)$ and $r(A)$ denote the numbers of occurrences of left and right parentheses in A , respectively. We prove by structural induction that

$$l(A) = r(A)$$

for every propositional formula A .

If $A = p_n$, then $l(A) = 0 = r(A)$. Suppose next that the claim holds for a formula A . Negation introduces no parentheses, so

$$l(\neg A) = l(A) = r(A) = r(\neg A).$$

Finally, suppose that the claim holds for formulas A and B , and let $\circ$ be any binary connective. The formula $(A \circ B)$ introduces one left and one right parenthesis. Hence

$$\begin{aligned} l((A \circ B)) &= l(A) + l(B) + 1 \\ &= r(A) + r(B) + 1 \\ &= r((A \circ B)). \end{aligned}$$

The result follows by structural induction. $\square$

2.2.2. Closures.

Definition 2.7 (Closure). Let X be a set, let $B \subseteq X$, and, for $1 \leq i \leq k$, let

$$f_i: X^{n_i} \longrightarrow X$$

be a function. A subset $Y \subseteq X$ is *closed under f_i* if

$$f_i(y_1, \dots, y_{n_i}) \in Y$$

whenever $y_1, \dots, y_{n_i} \in Y$. The *closure of B under $f_1, \dots, f_k$* , denoted by

$$\text{cl}(B; f_1, \dots, f_k),$$

is the smallest subset of X that contains B and is closed under every f_i .

Equivalently, the closure can be written as the intersection

$$\text{cl}(B; f_1, \dots, f_k) = \bigcap \{Y \subseteq X : B \subseteq Y \text{ and } Y \text{ is closed under every } f_i\}.$$

Example 2.8. Let $X = \mathbb{Z}$, $B = \{1\}$, and define functions $f, g: \mathbb{Z} \rightarrow \mathbb{Z}$ by

$$f(x) = x + 1 \quad \text{and} \quad g(x) = -x.$$

Then

$$\text{cl}(\{1\}; f, g) = \mathbb{Z}.$$

Indeed, every subset of $\mathbb{Z}$ that contains 1 and is closed under g must also contain -1 . Closure under f then gives 0, and repeated applications of f give every positive integer. Applying g to these integers gives every negative integer. Thus any such closed set is all of $\mathbb{Z}$, while $\mathbb{Z}$ itself clearly contains 1 and is closed under both functions.

Corollary 2.9 (Induction on a closure). *Set*

$$C = \text{cl}(B; f_1, \dots, f_k).$$

To prove that every element of C satisfies a property P , it suffices to prove that

- (1) *every element of B satisfies P ; and*
- (2) *for each i , if $x_1, \dots, x_{n_i}$ satisfy P , then $f_i(x_1, \dots, x_{n_i})$ also satisfies P .*

Proof. Let Y be the set of all elements of $\text{cl}(B; f_1, \dots, f_k)$ that satisfy P . The two assumptions imply that $B \subseteq Y$ and that Y is closed under every f_i . Since the closure is the smallest such set,

$$\text{cl}(B; f_1, \dots, f_k) \subseteq Y.$$

Hence every element of the closure satisfies P . $\square$

Let Σ be the alphabet of propositional logic, let Σ^* be the set of all finite strings over Σ , and let

$$V = \{p_n : n \in \mathbb{N}\}$$

be the set of propositional symbols. The logical connectives determine the following functions on strings:

$$f_{\neg}(A) = \neg A$$

and

$$f_{\circ}(A, B) = (A \circ B), \quad \circ \in \{\wedge, \vee, \rightarrow, \leftrightarrow\}.$$

The set of propositional formulas is precisely

$$\text{cl}(V; f_{\neg}, f_{\wedge}, f_{\vee}, f_{\rightarrow}, f_{\leftrightarrow}).$$

In particular, if $S \subseteq \Sigma^*$ contains every propositional symbol and is closed under all functions associated with the logical connectives, then S contains every propositional formula.

2.2.3. Semantics of propositional logic.

Definition 2.10 (Truth assignment). A *truth assignment* is a function

$$v: V \longrightarrow \{0, 1\},$$

where V is the set of propositional symbols. The values 0 and 1 represent **false** and **true**, respectively.

The truth values of the logical connectives are defined as follows.

Definition 2.11 (Negation). The negation $\neg A$ is true exactly when A is false.

A	$\neg A$
0	1
1	0

Definition 2.12 (Conjunction). The conjunction $(A \wedge B)$ is true exactly when both A and B are true.

A	B	$(A \wedge B)$
0	0	0
0	1	0
1	0	0
1	1	1

Definition 2.13 (Disjunction). The disjunction $(A \vee B)$ is true exactly when at least one of A and B is true.

A	B	$(A \vee B)$
0	0	0
0	1	1
1	0	1
1	1	1

Definition 2.14 (Implication). The implication $(A \rightarrow B)$ is false exactly when A is true and B is false. It is important to note that implication is **not** symmetric.

A	B	$(A \rightarrow B)$
0	0	1
0	1	1
1	0	0
1	1	1

Definition 2.15 (Equivalence). The equivalence $(A \leftrightarrow B)$ is true exactly when A and B have the same truth value.

A	B	$(A \leftrightarrow B)$
0	0	1
0	1	0
1	0	0
1	1	1

Definition 2.16 (Truth value). Let A be a propositional formula and let v be a truth assignment. The *truth value* $v(A)$ of A is defined by the following rules:

- (1) If A is a propositional symbol p_n , then $v(A)$ is already defined.
- (2) If $A = \neg B$ and $v(B)$ is defined, then

$$v(A) = \begin{cases} 0, & \text{if } v(B) = 1, \\ 1, & \text{if } v(B) = 0. \end{cases}$$

Another way to write this is $v(A) = 1 - v(B)$.

- (3) If $A = (B \wedge C)$ and $v(B)$ and $v(C)$ are defined, then $v(A) = 1$ exactly when $v(B) = v(C) = 1$, and $v(A) = 0$ otherwise. Another way to write this is $v(A)v(B)$.
- (4) If $A = (B \vee C)$ and $v(B)$ and $v(C)$ are defined, then $v(A) = 1$ when at least one of $v(B) = 1$ and $v(C) = 1$ holds, and $v(A) = 0$ when $v(B) = v(C) = 0$.
- (5) If $A = (B \rightarrow C)$ and $v(B)$ and $v(C)$ are defined, then $v(A) = 0$ when $v(B) = 1$ and $v(C) = 0$, and $v(A) = 1$ in all other cases.
- (6) If $A = (B \leftrightarrow C)$ and $v(B)$ and $v(C)$ are defined, then $v(A) = 1$ iff $v(B) = v(C)$.

Definition 2.17 (Classification of propositional formulas). A propositional formula A is

- (1) a *tautology* if $v(A) = 1$ for every truth assignment v ;
- (2) a *contradiction* if $v(A) = 0$ for every truth assignment v ;
- (3) *contingent* if there are truth assignments v and w such that $v(A) = 1$ and $w(A) = 0$;
- (4) *satisfiable* if $v(A) = 1$ for at least one truth assignment v ; and
- (5) *falsifiable* if $v(A) = 0$ for at least one truth assignment v .

Example 2.18 (De Morgan's laws). Consider the formulas

$$(\neg(A \wedge B) \leftrightarrow (\neg A \vee \neg B)) \quad \text{and} \quad (\neg(A \vee B) \leftrightarrow (\neg A \wedge \neg B)).$$

Their truth values can be compared using the following table:

A	B	$\neg(A \wedge B)$	$(\neg A \vee \neg B)$	$\neg(A \vee B)$	$(\neg A \wedge \neg B)$
0	0	1	1	1	1
0	1	1	1	0	0
1	0	1	1	0	0
1	1	0	0	0	0

The two sides of each equivalence have the same truth value under every truth assignment. Consequently, both equivalences are tautologies. These equivalences are known as *De Morgan's laws*.

2.3. 10.09.2026.

2.3.1. Logical implication and equivalence.

Definition 2.19 (Logical implication). A formula A *logically implies* a formula B , written

$$A \Rightarrow B,$$

if, for every truth assignment v , $v(A) = 1$ implies $v(B) = 1$. Equivalently, $A \Rightarrow B$ exactly when $(A \rightarrow B)$ is a tautology.

Definition 2.20 (Logical equivalence). Formulas A and B are *logically equivalent*, written

$$A \Leftrightarrow B,$$

if $v(A) = v(B)$ for every truth assignment v . Equivalently, $A \Leftrightarrow B$ exactly when $(A \leftrightarrow B)$ is a tautology.

The symbols $\Rightarrow$ and $\Leftrightarrow$ are used in the metalanguage to state relations between formulas; unlike $\rightarrow$ and $\leftrightarrow$, they are not connectives used to construct propositional formulas.

Definition 2.21 (Literal). A *literal* is either a propositional symbol p_n or its negation $\neg p_n$. The former is a *positive literal*, and the latter is a *negative literal*.

Definition 2.22 (Disjunctive normal form). A formula is in *disjunctive normal form* (DNF) if it is a disjunction of conjunctions of literals. In other words, it has the form

$$\bigvee_{i=1}^m \left(\bigwedge_{j=1}^{k_i} L_{ij} \right),$$

where $m \geq 1$, each $k_i \geq 1$, and every L_{ij} is a literal.

Definition 2.23 (Conjunctive normal form). A formula is in *conjunctive normal form* (CNF) if it is a conjunction of disjunctions of literals. In other words, it has the form

$$\bigwedge_{i=1}^m \left(\bigvee_{j=1}^{k_i} L_{ij} \right),$$

where $m \geq 1$, each $k_i \geq 1$, and every L_{ij} is a literal.

For a literal L , let $\bar{L}$ denote its *complementary literal*, defined by

$$\overline{p_n} = \neg p_n \quad \text{and} \quad \overline{\neg p_n} = p_n.$$

De Morgan's laws show that DNF and CNF are dual: negating a DNF formula gives an equivalent CNF formula, and negating a CNF formula gives an equivalent DNF formula. More precisely,

$$\neg \bigvee_{i=1}^m \left(\bigwedge_{j=1}^{k_i} L_{ij} \right) \Leftrightarrow \bigwedge_{i=1}^m \left(\bigvee_{j=1}^{k_i} \bar{L}_{ij} \right),$$

and the analogous equivalence holds with $\wedge$ and $\vee$ interchanged.

Definition 2.24 (Boolean function). An n -ary *truth function* is a function

$$f: \{0, 1\}^n \longrightarrow \{0, 1\}.$$

A propositional formula A whose propositional symbols are among $p_1, \dots, p_n$ determines the truth function

$$f_A(x_1, \dots, x_n) = v(A),$$

where $v(p_i) = x_i$ for every $1 \leq i \leq n$.

Theorem 2.25 (Normal form theorem). *Let f be an n -ary truth function, where $n \geq 1$. Then there are formulas A_D in DNF and A_C in CNF, containing only the propositional symbols $p_1, \dots, p_n$, such that*

$$f_{A_D} = f_{A_C} = f.$$

Consequently, every propositional formula is logically equivalent to a formula in DNF and to a formula in CNF.

Proof. We construct the formulas directly from the truth table of f .

To construct A_D , consider each row $a = (a_1, \dots, a_n) \in \{0, 1\}^n$ on which $f(a) = 1$, and define the formula

$$M_a = L_1^a \wedge \dots \wedge L_n^a, \quad L_i^a = \begin{cases} p_i, & \text{if } a_i = 1, \\ \neg p_i, & \text{if } a_i = 0. \end{cases}$$

Thus M_a is true under a truth assignment v exactly when $(v(p_1), \dots, v(p_n)) = a$.

If there is at least one such row, let

$$A_D = \bigvee_{\substack{a \in \{0,1\}^n \\ f(a)=1}} M_a.$$

Under an assignment corresponding to $b \in \{0, 1\}^n$, the formula A_D is true exactly when the disjunction contains M_b , which happens exactly when $f(b) = 1$. If f is constantly false, take $A_D = (p_1 \wedge \neg p_1)$. Thus $f_{A_D} = f$.

Dually, to construct A_C , consider each row a on which $f(a) = 0$ and define

$$C_a = K_1^a \vee \dots \vee K_n^a, \quad K_i^a = \begin{cases} p_i, & \text{if } a_i = 0, \\ \neg p_i, & \text{if } a_i = 1. \end{cases}$$

The clause C_a is false exactly on the row a . If there is at least one such row, let

$$A_C = \bigwedge_{\substack{a \in \{0,1\}^n \\ f(a)=0}} C_a.$$

This conjunction is true on a row b exactly when $f(b) = 1$. If f is constantly true, take $A_C = (p_1 \vee \neg p_1)$. Thus $f_{A_C} = f$.

All finite conjunctions and disjunctions above may be given any fixed binary parenthesization. Finally, for any propositional formula, list its distinct propositional symbols as $q_1, \dots, q_n$ and apply the same construction with q_i in place of p_i . This gives logically equivalent formulas in DNF and CNF. $\square$

Example 2.26 (Constructing normal forms from a truth function). Consider the binary truth function whose truth table is

x_1	x_2	$f(x_1, x_2)$
0	0	0
0	1	1
1	0	1
1	1	0

The rows where $f = 1$ are $(0, 1)$ and $(1, 0)$. Replace a 1 in a row by p_i , replace a 0 by $\neg p_i$, and join the literals in each row with $\wedge$. This gives the formulas

$$M_{(0,1)} = (\neg p_1 \wedge p_2) \quad \text{and} \quad M_{(1,0)} = (p_1 \wedge \neg p_2).$$

Finally, join the formulas for the selected rows with $\vee$:

$$A = ((\neg p_1 \wedge p_2) \vee (p_1 \wedge \neg p_2)).$$

This is a DNF formula with $f_A = f$.

To construct a CNF formula, use instead the rows $(0, 0)$ and $(1, 1)$, where $f = 0$. The clauses that are false on exactly these rows are

$$C_{(0,0)} = (p_1 \vee p_2) \quad \text{and} \quad C_{(1,1)} = (\neg p_1 \vee \neg p_2).$$

Their conjunction

$$B = ((p_1 \vee p_2) \wedge (\neg p_1 \vee \neg p_2))$$

is in CNF and satisfies $f_B = f$. Thus $A \Leftrightarrow B$.

2.4. 14.09.2026.

2.4.1. Universal sets of connectives.

Definition 2.27 (Universal set of connectives). A set C of logical connectives is *universal*, or *functionally complete*, if, for every $n \geq 1$ and every truth function

$$f: \{0, 1\}^n \longrightarrow \{0, 1\},$$

there is a formula A , containing only the propositional symbols $p_1, \dots, p_n$ and connectives from C , such that $f_A = f$.

Theorem 2.28. *The set $\{\neg, \wedge\}$ is universal.*

Proof. Let $f: \{0, 1\}^n \rightarrow \{0, 1\}$ be a truth function. The DNF construction gives a DNF formula A such that $f_A = f$. We prove by structural induction that every formula built from literals using $\wedge$ and $\vee$ is logically equivalent to a formula containing only $\neg$ and $\wedge$.

A literal is either p_i or $\neg p_i$, so the claim holds in the base case. Suppose that the induction hypothesis gives formulas B^* and C^* , containing only $\neg$ and $\wedge$, such that $B^* \Leftrightarrow B$ and $C^* \Leftrightarrow C$. Then

$$(B \wedge C) \Leftrightarrow (B^* \wedge C^*)$$

and, by De Morgan's law,

$$(B \vee C) \Leftrightarrow \neg(\neg B^* \wedge \neg C^*).$$

Both formulas on the right use only $\neg$ and $\wedge$. Structural induction therefore gives a formula A^* , containing only these connectives, such that $A^* \Leftrightarrow A$. Hence $f_{A^*} = f_A = f$, so $\{\neg, \wedge\}$ is universal. $\square$

Theorem 2.29. *The set $\{\wedge, \vee\}$ is not universal.*

Proof. We prove by structural induction that every formula A containing only $\wedge$ and $\vee$ is false under the truth assignment v for which $v(p_i) = 0$ for every propositional symbol p_i .

If $A = p_i$, then $v(A) = 0$. Suppose that the claim holds for formulas B and C . Then

$$v(B \wedge C) = 0 \quad \text{and} \quad v(B \vee C) = 0,$$

so the claim also holds for formulas formed using either allowed connective. It therefore holds for every formula containing only $\wedge$ and $\vee$.

The unary truth function $f(x) = 1 - x$ has value $f(0) = 1$, so no such formula can represent it. Hence $\{\wedge, \vee\}$ is not universal. $\square$

2.5. 21.09.2026.

2.5.1. Clauses and clause sets.

Definition 2.30 (Clause). A *clause* is a finite set of literals. It represents the disjunction of its members. For a truth assignment v , define

$$v(C) = 1 \quad \text{if and only if} \quad v(q) = 1 \text{ for some } q \in C,$$

and set $v(C) = 0$ otherwise. In particular, the *empty clause* $\emptyset$ is false under every truth assignment.

Definition 2.31 (Clause set). A *clause set* $\mathcal{C}$ is a set of clauses. Define

$$v(\mathcal{C}) = 1 \quad \text{if and only if} \quad v(C) = 1 \text{ for every } C \in \mathcal{C},$$

and set $v(\mathcal{C}) = 0$ otherwise. Thus a finite clause set represents the conjunction of its clauses. It is *satisfiable* if some truth assignment makes it true, and *unsatisfiable* otherwise.

Remark 2.32. The empty clause and the empty clause set have different meanings: the empty clause is always false, whereas the empty clause set is always true (the condition on all its members holds vacuously). In particular, $\{\emptyset\}$ is an unsatisfiable clause set. A clause containing both p_i and $\neg p_i$ is always true.

Example 2.33 (Encoding CNF as a clause set). Since $(p_0 \rightarrow p_1) \Leftrightarrow (\neg p_0 \vee p_1)$, the formula

$$p_0 \wedge (p_0 \rightarrow p_1)$$

is represented by $\mathcal{C} = \{\{p_0\}, \{\neg p_0, p_1\}\}$. The literals within a clause are joined by disjunction, and the clauses within a clause set are joined by conjunction. Order and repetitions do not matter in these set representations.

2.5.2. The resolution rule and resolution proofs.

Definition 2.34 (Resolution rule). The *resolution rule* is

$$\frac{C_1 \cup \{p_i\} \quad C_2 \cup \{\neg p_i\}}{C_1 \cup C_2}.$$

The conclusion is called a *resolvent*, and p_i is the *pivot*. In the usual application to clauses D, E with $p_i \in D$ and $\neg p_i \in E$, take $C_1 = D \setminus \{p_i\}$ and $C_2 = E \setminus \{\neg p_i\}$. Only the selected complementary pair is removed.

Example 2.35 (Resolution steps). Modus ponens is the resolution step

$$\frac{\{p_0\} \quad \{\neg p_0, p_1\}}{\{p_1\}}.$$

From the same two clauses one may obtain different resolvents by choosing different pivots. For example,

$$\frac{\{p_0, p_1\} \quad \{\neg p_0, \neg p_1\}}{\{p_0, \neg p_0\}} \quad \text{or} \quad \frac{\{p_0, p_1\} \quad \{\neg p_0, \neg p_1\}}{\{p_1, \neg p_1\}}.$$

Both conclusions are tautological clauses. Removing both complementary pairs at once to obtain $\emptyset$ would be invalid: the premises are simultaneously true when $v(p_0) = 1$ and $v(p_1) = 0$.

Definition 2.36 (Resolution proof). A *resolution proof* of a clause C from a clause set $\mathcal{C}$ is a finite sequence $C_0, \dots, C_m$, where $m \geq 0$, such that $C_m = C$ and, for every $i \leq m$, either

- (1) $C_i \in \mathcal{C}$, or
- (2) C_i follows by resolution from C_k, C_l for some $k, l < i$.

We write $\mathcal{C} \vdash_R C$ if such a proof exists. A proof of $\emptyset$ is called a *resolution refutation*.

2.5.3. Soundness of resolution.

Lemma 2.37 (Soundness of the resolution rule). If $v(C_1 \cup \{p_i\}) = v(C_2 \cup \{\neg p_i\}) = 1$, then $v(C_1 \cup C_2) = 1$.

Proof. If $v(p_i) = 1$, then $v(\neg p_i) = 0$. Since the second premise is true, some literal of C_2 must be true. Hence $v(C_1 \cup C_2) = 1$. If $v(p_i) = 0$, truth of the first premise similarly implies that some literal of C_1 is true, so again $v(C_1 \cup C_2) = 1$. $\square$

Theorem 2.38 (Soundness). If $\mathcal{C} \vdash_R C$, then every truth assignment satisfying $\mathcal{C}$ satisfies C . Thus every resolution conclusion is a logical consequence of its assumptions.

Proof. Let $C_0, \dots, C_m = C$ be a resolution proof from $\mathcal{C}$, and let $v(\mathcal{C}) = 1$. We prove $v(C_i) = 1$ by strong induction on i . For $i = 0$, there are no earlier lines, so $C_0 \in \mathcal{C}$ and $v(C_0) = 1$. Suppose $i > 0$ and $v(C_j) = 1$ for every $j < i$. If $C_i \in \mathcal{C}$, it is true by assumption. Otherwise it is a resolvent of C_k, C_l with $k, l < i$. Both premises are true by the induction hypothesis, and the soundness lemma gives $v(C_i) = 1$. In particular, $v(C_m) = v(C) = 1$. $\square$

Remark 2.39 (Completeness versus refutation completeness). Resolution does not derive every logical consequence: from $\{\{p_0\}\}$ one cannot derive $\{p_0, p_1\}$, although $p_0 \Rightarrow (p_0 \vee p_1)$. Indeed, the only available clause is $\{p_0\}$, and no resolution step is possible without a complementary literal. Resolution is nevertheless *refutation complete*: an unsatisfiable clause set admits a derivation of the empty clause.

2.5.4. Proving an implication by refutation. To prove $A \Rightarrow B$, it suffices to show that $A \wedge \neg B$ is unsatisfiable: a truth assignment falsifying the implication would make both A and $\neg B$ true. Convert $A \wedge \neg B$ into CNF, encode it as a clause set, and derive $\emptyset$.

Example 2.40. We prove

$$(p_0 \wedge (p_0 \rightarrow p_1)) \Rightarrow (p_1 \vee p_2).$$

Negating the conclusion gives $\neg(p_1 \vee p_2) \Leftrightarrow (\neg p_1 \wedge \neg p_2)$. Hence the premises together with the negated conclusion give

$$\mathcal{C} = \{\{p_0\}, \{\neg p_0, p_1\}, \{\neg p_1\}, \{\neg p_2\}\}.$$

A resolution refutation is

0. $\{p_0\}$ assumption,
1. $\{\neg p_0, p_1\}$ assumption,
2. $\{\neg p_1\}$ assumption,
3. $\{\neg p_2\}$ assumption,
4. $\{p_1\}$ resolution of lines 0 and 1,
5. $\emptyset$ resolution of lines 4 and 2.

Thus $\mathcal{C} \vdash_R \emptyset$, so soundness implies that $\mathcal{C}$ is unsatisfiable and proves the implication. Line 3 need not be used: the premises already imply p_1 .

2.5.5. Refutation completeness.

Theorem 2.41 (Finite refutation completeness). *If $\mathcal{C}$ is a finite unsatisfiable clause set, then $\mathcal{C} \vdash_R \emptyset$.*

Proof. We use strong induction on the number n of distinct propositional symbols occurring in $\mathcal{C}$. This number is finite because both $\mathcal{C}$ and each of its clauses are finite.

If $n = 0$, the only possible clause is $\emptyset$. Since the empty clause set is satisfiable, unsatisfiability forces $\emptyset \in \mathcal{C}$. The one-line proof ($\emptyset$) suffices.

Suppose $n > 0$ and the result holds for fewer than n symbols. Choose a symbol p_i occurring in $\mathcal{C}$, and partition the clause set as in the book:

$$\begin{aligned}\mathcal{C}_{00} &= \{C \in \mathcal{C} : p_i \notin C, \neg p_i \notin C\}, \\ \mathcal{C}_{01} &= \{C \in \mathcal{C} : p_i \notin C, \neg p_i \in C\}, \\ \mathcal{C}_{10} &= \{C \in \mathcal{C} : p_i \in C, \neg p_i \notin C\}, \\ \mathcal{C}_{11} &= \{C \in \mathcal{C} : p_i \in C, \neg p_i \in C\}.\end{aligned}$$

Every clause in $\mathcal{C}_{11}$ is tautological. Form all resolvents between the negative and positive parts on the pivot p_i :

$$\mathcal{C}_{01} \times_R \mathcal{C}_{10} = \{(C_1 \setminus \{\neg p_i\}) \cup (C_2 \setminus \{p_i\}) : C_1 \in \mathcal{C}_{01}, C_2 \in \mathcal{C}_{10}\}.$$

Define

$$\mathcal{C}' = \mathcal{C}_{00} \cup (\mathcal{C}_{01} \times_R \mathcal{C}_{10}).$$

No clause of $\mathcal{C}'$ contains p_i or $\neg p_i$, so $\mathcal{C}'$ contains at most $n - 1$ distinct symbols.

Claim: $\mathcal{C}'$ is unsatisfiable. Suppose instead that $v(\mathcal{C}') = 1$. We show that changing only the value of p_i , if necessary, gives a satisfying assignment for $\mathcal{C}$. There are two cases.

- (1) Suppose $v(C \setminus \{\neg p_i\}) = 1$ for every $C \in \mathcal{C}_{01}$. Define $w(p_i) = 1$ and $w(p_j) = v(p_j)$ for $j \neq i$. Each clause of $\mathcal{C}_{01}$ remains true because its part not involving p_i is true. Each clause of $\mathcal{C}_{10}$ is true because it contains p_i .
- (2) Otherwise, choose $C_1 \in \mathcal{C}_{01}$ such that $v(C_1 \setminus \{\neg p_i\}) = 0$. For every $C_2 \in \mathcal{C}_{10}$, the resolvent

$$(C_1 \setminus \{\neg p_i\}) \cup (C_2 \setminus \{p_i\})$$

belongs to $\mathcal{C}'$ and is true under v . Its first part is false, so $v(C_2 \setminus \{p_i\}) = 1$. Define $w(p_i) = 0$ and $w(p_j) = v(p_j)$ for $j \neq i$. Every clause of $\mathcal{C}_{01}$ is true because it contains $\neg p_i$; every clause of $\mathcal{C}_{10}$ is true because its remaining part is true.

In both cases, the clauses of $\mathcal{C}_{00}$ remain true because their truth values do not depend on p_i , and all clauses of $\mathcal{C}_{11}$ are true under every assignment. Thus $w(\mathcal{C}) = 1$, contradicting the unsatisfiability of $\mathcal{C}$. This proves the claim.

By the induction hypothesis, $\mathcal{C}' \vdash_R \emptyset$. To obtain a proof from $\mathcal{C}$, first list the clauses of $\mathcal{C}$ as assumptions and derive all clauses in $\mathcal{C}_{01} \times_R \mathcal{C}_{10}$ by resolution. Every assumption used in the refutation from $\mathcal{C}'$ is now available: it is either in $\mathcal{C}_{00} \subseteq \mathcal{C}$ or among these resolvents. Reproduce the resolution steps of that refutation, referring to the already available lines whenever it uses an assumption. This yields a finite resolution proof of $\emptyset$ from $\mathcal{C}$. $\square$

Remark 2.42 (Scope of the proof). The course book states Lemma 6.7 for arbitrary clause sets, but the induction above proves the finite case treated in the lecture. The general case follows using propositional compactness: every unsatisfiable clause set has a finite unsatisfiable subset, whose refutation is also a refutation from the original set. Compactness is an additional result; it is not proved by this induction.

Corollary 2.43 (Refutation criterion). *For a finite clause set $\mathcal{C}$,*

$$\mathcal{C} \text{ is unsatisfiable} \iff \mathcal{C} \vdash_R \emptyset.$$

Proof. The forward direction is refutation completeness. Conversely, if $\mathcal{C} \vdash_R \emptyset$, soundness says that any assignment satisfying $\mathcal{C}$ must satisfy $\emptyset$. Since the empty clause is always false, no such assignment exists. $\square$

2.6. 28.09.2026.

2.6.1. *Natural deduction and assumptions.* Natural deduction gives introduction and elimination rules for each logical connective. An introduction rule tells us how to prove a formula with that connective; an elimination rule tells us what we can infer from such a formula. Each rule has one to three premises and exactly one conclusion. Proofs are drawn as trees, with premises above a horizontal line and the conclusion below it.

Definition 2.44 (Derivability). Let Γ be a set of propositional formulas. We write

$$\Gamma \vdash A$$

if there is a natural deduction proof of A whose undischarged assumptions all belong to Γ . We write $\vdash A$ for $\emptyset \vdash A$: a proof with no undischarged assumptions.

An assumption may be used more than once. Some rules introduce a temporary assumption and later *discharge* it. We mark the relevant occurrences as $[A]^i$ and put the same label i beside the rule that discharges them. Below that rule, these occurrences are no longer open assumptions. Other assumptions remain open. The notation

$$\begin{array}{c} [A]^i \\ \vdots \\ B \end{array}$$

stands for a derivation of B under the temporary assumption A , possibly together with other open assumptions. A derivation may consist of just an assumption, with no inference steps.

We retain the lecture's labels: T means introduction (Finnish *tuonti*) and E means elimination.

2.6.2. Conjunction.

Introduction. From proofs of both conjuncts, infer their conjunction:

$$\frac{A \quad B}{A \wedge B} \wedge T.$$

Elimination. From a conjunction, infer either conjunct:

$$\frac{A \wedge B}{A} \wedge E \qquad \frac{A \wedge B}{B} \wedge E.$$

Example 2.45 (Commutativity of conjunction). We prove $\{A \wedge B\} \vdash B \wedge A$:

$$\frac{\frac{A \wedge B}{B} \wedge E \quad \frac{A \wedge B}{A} \wedge E}{B \wedge A} \wedge T$$

Both leaves use the same open assumption $A \wedge B$.

2.6.3. *Disjunction introduction.* From either disjunct, infer the disjunction:

$$\frac{A}{A \vee B} \vee\text{T} \qquad \frac{B}{A \vee B} \vee\text{T}.$$

Example 2.46 (From a conjunction to a disjunction). There are two proofs of $\{A \wedge B\} \vdash A \vee B$:

$$\frac{\frac{A \wedge B}{A} \wedge\text{E}}{A \vee B} \vee\text{T} \qquad \text{or} \qquad \frac{\frac{A \wedge B}{B} \wedge\text{E}}{A \vee B} \vee\text{T}.$$

2.6.4. *Implication elimination.* Implication elimination is *modus ponens*: from $A \rightarrow B$ and A , infer B .

$$\frac{A \rightarrow B \quad A}{B} \rightarrow\text{E}.$$

Example 2.47 (Two applications of modus ponens). We prove $\{A \wedge B, A \rightarrow (B \rightarrow C)\} \vdash C$:

$$\frac{A \rightarrow (B \rightarrow C) \quad \frac{\frac{A \wedge B}{A} \wedge\text{E}}{B \rightarrow C} \rightarrow\text{E}}{C} \rightarrow\text{E} \quad \frac{\frac{A \wedge B}{B} \wedge\text{E}}{C} \rightarrow\text{E}$$

First extract A and B from the conjunction. Apply modus ponens with A to obtain $B \rightarrow C$, and then with B to obtain C .

2.6.5. *Equivalence and negation elimination.*

Equivalence elimination. An equivalence can be used in either direction:

$$\frac{A \leftrightarrow B \quad A}{B} \leftrightarrow\text{E} \qquad \frac{A \leftrightarrow B \quad B}{A} \leftrightarrow\text{E}.$$

Negation elimination. The lecture uses double negation elimination:

$$\frac{\neg\neg A}{A} \neg\text{E}.$$

This is a classical rule: a proof of $\neg\neg A$ allows us to conclude A .

2.6.6. *Disjunction elimination: proof by cases.* Suppose that $A \vee B$ has been proved. If we can derive the same conclusion C under the assumption A and under the assumption B , we may conclude C and discharge the two case assumptions:

$$\frac{\begin{array}{cc} [A]^i & [B]^j \\ A \vee B & \\ \vdots & \vdots \\ C & C \end{array}}{C} \vee\text{E}, i, j.$$

The two cases are separate subproofs. An assumption made only in one case cannot be used in the other case. Any other open assumptions in the subproofs remain open after this rule.

Example 2.48 (A common conjunct in both cases). We prove $\{(A \wedge C) \vee (B \wedge C)\} \vdash C$:

$$\frac{(A \wedge C) \vee (B \wedge C) \quad \frac{\frac{[A \wedge C]^1}{C} \wedge\text{E}}{C} \vee\text{E}, 1, 2}{C} \vee\text{E}, 1, 2$$

In either case, conjunction elimination gives C . The case assumptions are discharged; the original disjunction remains open.

Example 2.49 (Commutativity of disjunction). We prove $\{A \vee B\} \vdash B \vee A$:

$$\frac{A \vee B \quad \frac{[A]^1}{B \vee A} \vee T \quad \frac{[B]^2}{B \vee A} \vee T}{B \vee A} \vee E, 1, 2$$

Example 2.50 (Repeated disjuncts). We prove $\{A \vee A\} \vdash A$:

$$\frac{A \vee A \quad [A]^1 \quad [A]^2}{A} \vee E, 1, 2$$

Each case already has A as its assumption, so no inference is needed within either case.

2.6.7. *Implication introduction.* To prove $A \rightarrow B$, assume A , derive B , and discharge the assumption A :

$$\frac{\begin{array}{c} [A]^i \\ \vdots \\ B \end{array}}{A \rightarrow B} \rightarrow T, i.$$

The derivation of B may also use other assumptions; these remain open. It is permitted that the temporary assumption is never used, in which case the discharge is called *vacuous*.

Example 2.51 (Introducing a disjunct in the consequent). We prove $\emptyset \vdash A \rightarrow (A \vee B)$:

$$\frac{\frac{[A]^1}{A \vee B} \vee T}{A \rightarrow (A \vee B)} \rightarrow T, 1$$

Discharging A leaves no open assumptions.

Example 2.52 (An unused temporary assumption). We prove $\{A\} \vdash B \rightarrow A$:

$$\frac{A}{B \rightarrow A} \rightarrow T$$

The conclusion A is already available as an open assumption, so the temporary assumption B need not be used. Its discharge is vacuous; the assumption A remains open.

Example 2.53 (Identity). We prove $\vdash A \rightarrow A$:

$$\frac{[A]^1}{A \rightarrow A} \rightarrow T, 1$$

The one-formula derivation of A from itself suffices. Discharging it leaves no open assumptions.

2.6.8. *Equivalence introduction.* To prove $A \leftrightarrow B$, derive B assuming A , and derive A assuming B . Discharge both temporary assumptions:

$$\frac{\begin{array}{cc} [A]^i & [B]^j \\ \vdots & \vdots \\ B & A \end{array}}{A \leftrightarrow B} \leftrightarrow T, i, j.$$

Example 2.54 (Additional example: equivalence introduction). We prove $\vdash (A \wedge B) \leftrightarrow (B \wedge A)$. Each direction uses conjunction elimination followed by introduction:

$$\frac{\frac{[A \wedge B]^1}{B} \wedge E \quad \frac{[A \wedge B]^1}{A} \wedge E \quad \frac{[B \wedge A]^2}{A} \wedge E \quad \frac{[B \wedge A]^2}{B} \wedge E}{\frac{B \wedge A}{(A \wedge B) \leftrightarrow (B \wedge A)} \wedge T, 1, 2}$$

The final rule discharges both occurrences labelled 1 and both occurrences labelled 2. There are no open assumptions.

2.6.9. *Negation introduction.* To prove $\neg A$, assume A and derive a contradiction $B \wedge \neg B$. Then discharge the assumption A :

$$\frac{\begin{array}{c} [A]^i \\ \vdots \\ B \wedge \neg B \end{array}}{\neg A} \neg T, i.$$

The formula B can be any formula; it need not be the same as A .

Example 2.55 (Additional example: negation introduction (modus tollens)). We prove $\{A \rightarrow B, \neg B\} \vdash \neg A$. Assume A . Modus ponens gives B , which contradicts the premise $\neg B$:

$$\frac{\frac{A \rightarrow B \quad [A]^1}{B} \rightarrow E \quad \neg B}{\frac{B \wedge \neg B}{\neg A} \wedge T, 1} \neg T, 1$$

Only A is discharged. The premises $A \rightarrow B$ and $\neg B$ remain open.